

目 录

[示教器](#)

[快速开始](#)

[二次开发API](#)

[示例](#)

[硬件接口](#)

[自定义指令](#)

示教器

简介

示教器二次开发是基于示教软件的静态库所提供的接口来进行的二次开发。

二次开发静态库支持 windows linux arm-linux(T30 示教器)三种系统软件的开发：

基于 windows 版本可以开发上位机程序；

基于 linux 版本的二次开发静态库开发的程序可以用于调试或者生成基于 linux 系统的上位机软件；

基于 arm-linux(T30)版本的二次开发静态库生成的程序仅支持运行在本公司 T30 示教器。

提供完整的示教器基本功能，用户可以自行定制所需的工艺界面，提供用户自定义的指令，自定义界面的插入，提供通用的数据收发接口（通信数据格式统一为 JSON 格式）。

学习路线

在学习示教器的二次开发前，我们默认您已经具备以下知识：

1. C++语言；
2. QT 使用；
3. Linux 系统（Ubuntu 发行版）的使用；
4. 工业机器人相关基础知识；
5. QtCreator 编辑器。如果您不具备以上知识，我们给您推荐以下教程：

- [C++教程 - Qt 教程](#)
- [Ubuntu 完全教程，让你成为 Ubuntu 高手！](#)
- [Linux Ubuntu 基本操作指令](#)
- [纳博特小课堂](#)
- [纳博特控制系统操作手册](#)

快速开始

1. 环境安装

1.1 系统要求

当前推荐的开发环境操作系统为：

- Ubuntu 20.04

当前支持的 Qt 编译套件包括：

- Desktop Qt 5.9.0 GCC 64 bit
- LYX （示教盒嵌入式编译套件）

需要根据不同的目标平台，选择不同的 Qt 编译套件。

如果只需要在 Ubuntu 20.04 上运行，选择 Desktop Qt 5.9.0 GCC 64 bit 编译套件，如果需要在嵌入式示教盒上运行，选择 LYX 编译套件。

我们提供打包了所有开发环境的镜像文件，当然也可以在任意一台 Ubuntu 20.04 下参照小节《1.3 编译套件安装》来安装。

直接获取安装好了的虚拟机

[点击这里](#) 可以下载得到配置好开发环境的、基于VirtualBox软件使用的Ubuntu 20.04.1 系统的虚拟机；

[点击这里](#) 可以下载得到基于Windows系统的VirtualBox软件安装包，如需其他系统版本的请自行到 [VirtualBox官网](#) 进行下载。

- 安装VirtualBox软件（注意按转路径不能带有中文）
- 打开下载的“开发环境虚拟机”文件夹，按照“README.txt”文件说明安装虚拟机；
- 开机后登录账户“nbt”，密码是“123”。

1.2 Qt 下载

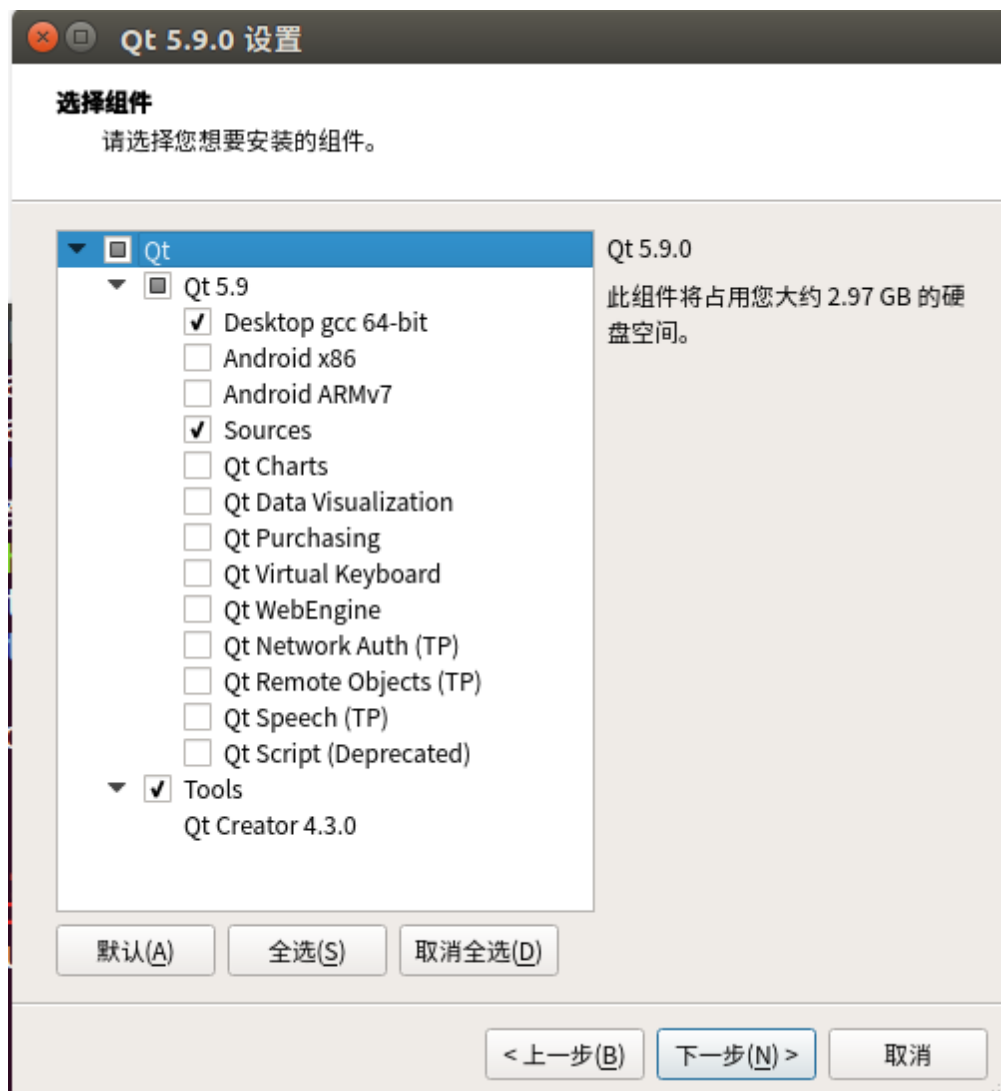
我们需要进入Qt的官网 [Qt Downloads](#) 选择 qt-opensource-linux-x64-5.9.0.run 下载。

-  推荐复制这个链接 https://download.qt.io/new_archive/qt/5.9/5.9.0/qt-opensource-linux-x64-5.9.0.run，打开迅雷下载速度更快。

下载完成后在Ubuntu 20.04中，打开cmd，运行：

```
1 sudo chmod a+x qt-opensource-linux-x64-5.9.0.run
2 sudo ./qt-opensource-linux-x64-5.9.0.run
```

出现qt安装程序后一直选择默认，直到这一步需要勾选如图所示的选项：



步骤全部完成后，Qt会被安装到/opt目录下。

启动Qt，运行：

```
1 /opt/Qt5.9.0/Tools/QtCreator/bin/qtcreator
```

1.3 编译套件安装

以下编译链根据具体的目标平台选择安装：

1.3.1 Desktop Qt 5.9.0 GCC 64 bit 编译套件安装

安装GCC，运行：

```
1 sudo apt install gcc-multilib
2 sudo apt install g++-multilib
3 sudo apt install lib32z1
4 sudo apt install libgl1-mesa-dev
```

1.3.2 LYX 编译套件安装

tslib.tar.gz	57.75 KB	2025-05-26 15:29
qtenv.tar.gz	35.51 MB	2025-05-26 15:29
gcc.tar.gz	86.72 MB	2025-05-26 15:30

将gcc.tar.gz和qtenv.tar.gz解压到系统的/opt/目录下

```
1 sudo tar -zxvf gcc.tar.gz -C /opt
2 sudo tar -zxvf qtenv.tar.gz -C /opt
```

将tslib.tar.gz解压到系统的/usr/local目录下

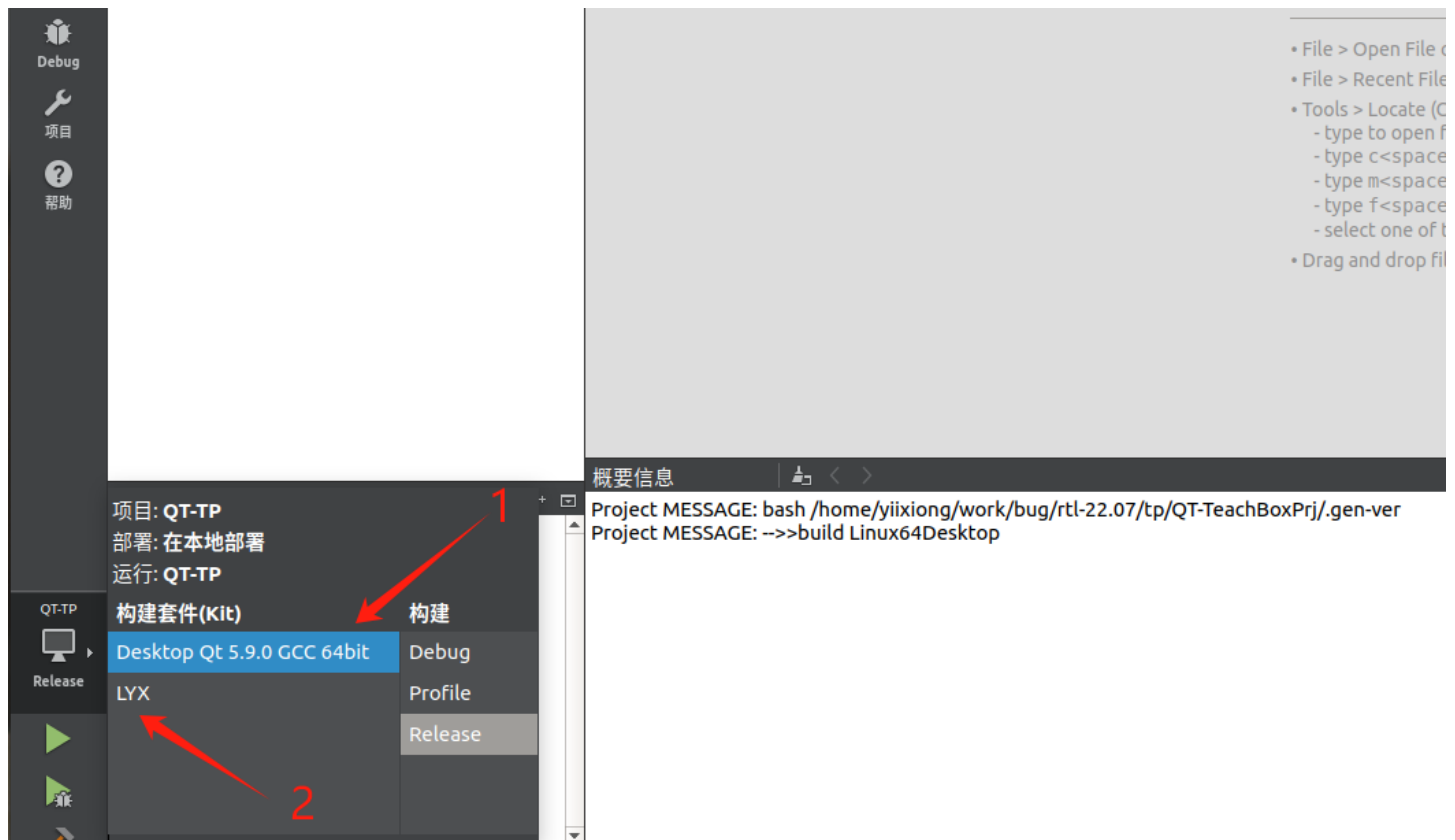
```
1 sudo tar -zxvf tslib.tar.gz -C /usr/local
```

配置32位编译环境

```
1 sudo dpkg --add-architecture i386
2 sudo apt update
3 sudo apt install libc6-dev-i386
4 sudo apt install g++-multilib
5 sudo apt install libstdc++6:i386
```

打开qt，添加 LYX 编译套件，详细步骤可见附件：

Qt开发环境配置说明.pdf	2.63 MB	2025-05-26 16:19
--------------------------------	---------	------------------



至此，我们常用的两个编译套件就配置完成了。

3. 远程登陆到示教器

3.1 确认电脑连接上示教器

首先确保你的电脑已经跟控制器连接了通讯，电脑的IP需要跟控制器的IP在同一网段下。示教器的出厂默认IP为192.168.1.245，所以电脑连接控制器的网口同样需要设置为1网段（比如可以设置192.168.1.110）。设置完成之后在Ubuntu中打开一个新的终端输入

```
1 ping 192.168.1.245
```

如出现不成功的情况，请检查网线的连接，电脑的IP，示教器的IP。

3.2 使用ssh登陆到示教器

```
1 ssh root@192.168.1.245
```

然后密码输入1234，即可进行远程控制了。

示教器程序执行在/userfs/app/下，使用./Qt-tp -qws即可启动示教器程序，使用killall -9 Qt-tp杀死示教器程序。

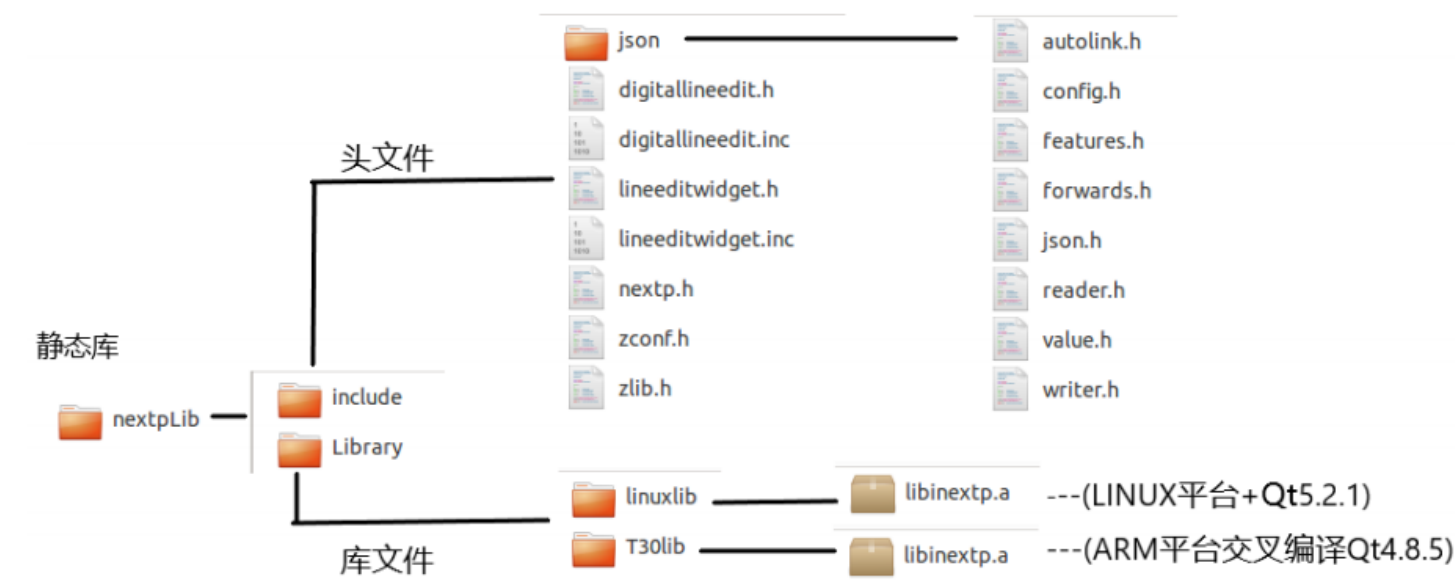
二次开发API

二次开发静态库简介

功能介绍

提供完整的示教器功能，支持添加用户自定义界面，支持特定字段的通信与控制系统通信。

静态库目录结构



nextpLib 为总文件夹；

Include 文件夹中为头文件；

Library 文件夹中为静态库文件包括 linux 平台和 ARM 平台的库文件（使用ARM 平台交叉编译的程序只适用于纳博特公司的 T30 示教器使用）；

静态库结构说明

1.nextp.h 头文件接口说明：

```
1 //创建 Nextp 类对象
2 static QPointer<Nextp> getInstance();
3 //获取系统字体
4 QString getSystemFont();
5 //用户自定义窗体传输到主程序
6 void setWidgetParentLocation(QPointer <QWidget> widget);
7 //向控制器发送消息
8 void sendMessage(const quint16 &command,const QByteArray &data);
9 //通知示教程序自定义窗体已经打开
10 widgetShowFinish()
11 //接收控制器消息信号
12 void signal_receiveMessage(const quint16 &command,const QByteArray &data);
13 //打开自定义窗体信号
14 void signal_openWidget();
15 //关闭自定义窗体信号
16 void signal_closeWidget();
17 //隐藏工艺主界面中除了自定义按钮的其他按钮
18 void hideTechnologyToolbuttons();
19 //静态库支持与控制器通信命令字
20 enum CommandList
21 {
22     SetFirstUserParaCommand = 0x9200,
23     GetFirstUserCommand = 0x9201,
24     ReceivedFirstUserCommand = 0x9202,
25     SetSecondUserParaCommand = 0x9203,
26     GetSecondUserCommand = 0x9204,
27     ReceivedSecondUserCommand = 0x9205,
28     SetThirdUserParaCommand = 0x9206,
29     GetThirdUserCommand = 0x9207,
30     ReceivedThirdUserCommand = 0x9208,
31     ReceivedThirdUserCommand = 0x9208,
32     SetFourthUserParaCommand = 0x9209,
33     GetFourthUserCommand = 0x920a,
34     ReceivedFourthUserCommand = 0x920b,
35     SetFifthUserParaCommand = 0x920c,
36     GetFifthUserCommand = 0x920d,
```



```
37     ReceivedFifthUserCommand = 0x920e,  
38 };
```

2.json/json.h 头文件提供 JSON 数据格式的组装和解析

组装 json 数据示例:

```
1  Json::Value root;  
2  Json::FastWriter wt;  
3  root["robot"] =1;  
4  root["booldata"] =true;  
5  root["data"] = 1.1;  
6  root["name"] ="nihao";
```

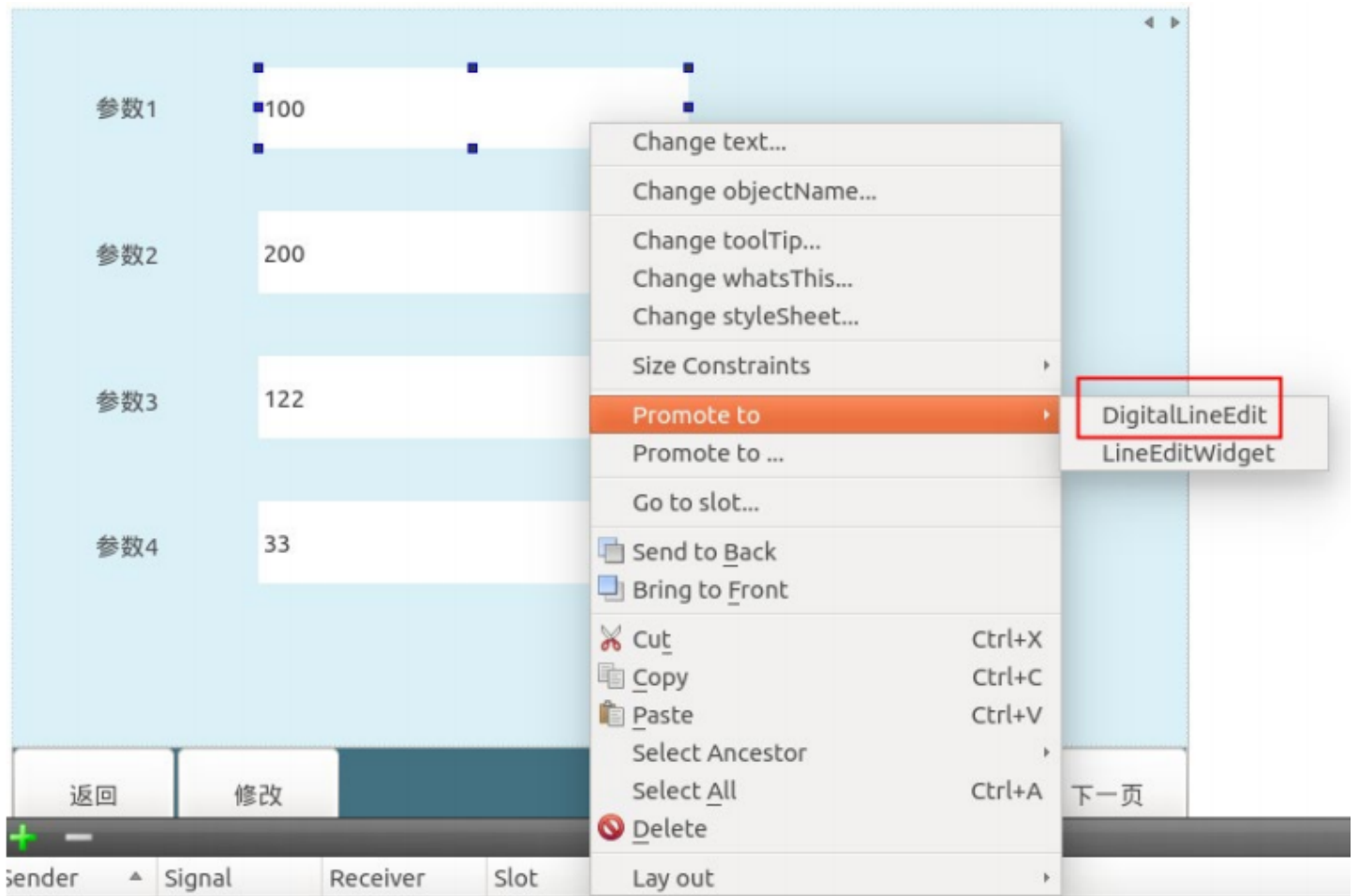
解析 Json 数据示例

```
1  QByteArray jsonData //控制器发送的数据  
2  Json::Value root;  
3  Json::Reader reader;  
4  QString jsonData = param.data();  
5  if(reader.parse(jsonData.toStdString(), root))  
6  {  
7      int robot = root["robot"].asInt();  
8      bool booldata= root["booldata"].asBool();  
9      Int data= root["data"].asDouble();  
10     Std::string name = root["name "].asString();  
11 }
```

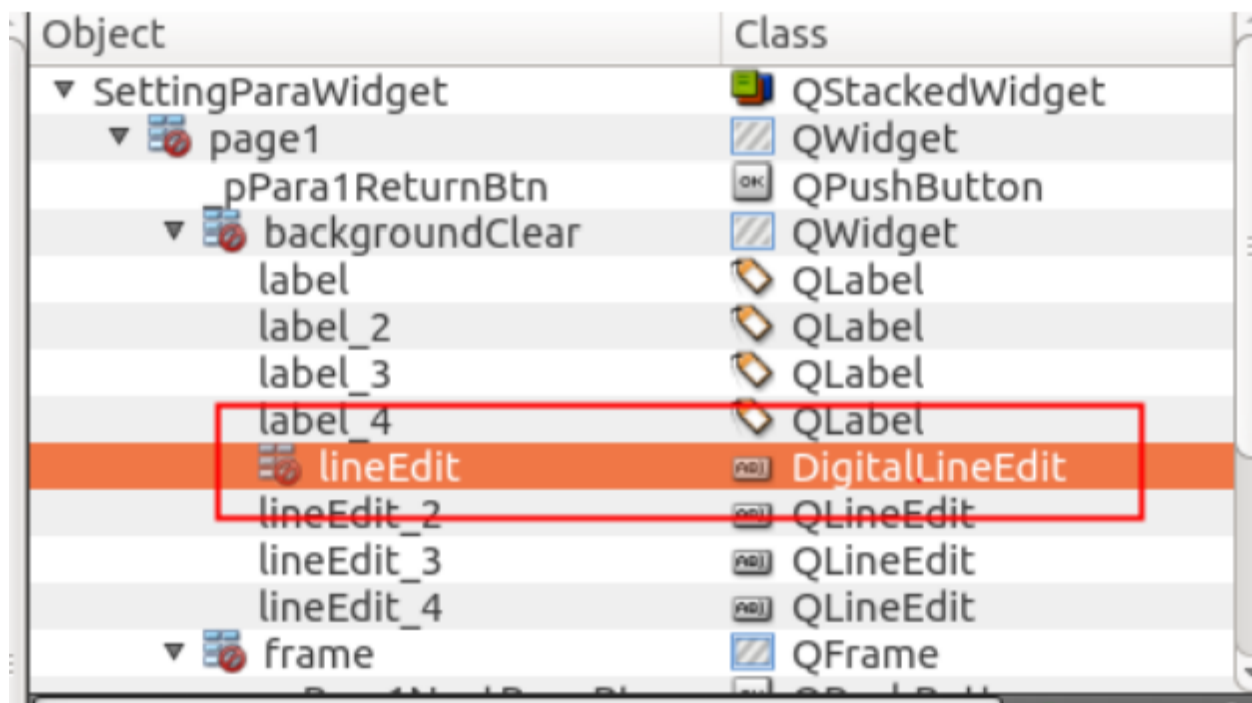
3.digitallineedit.h 提供数字输入框

支持将 QLineEdit 控件提升为数字输入框

提升方法：右键选择一个 QLineEdit 控件 --->Promote to--->DigitalLineEdit



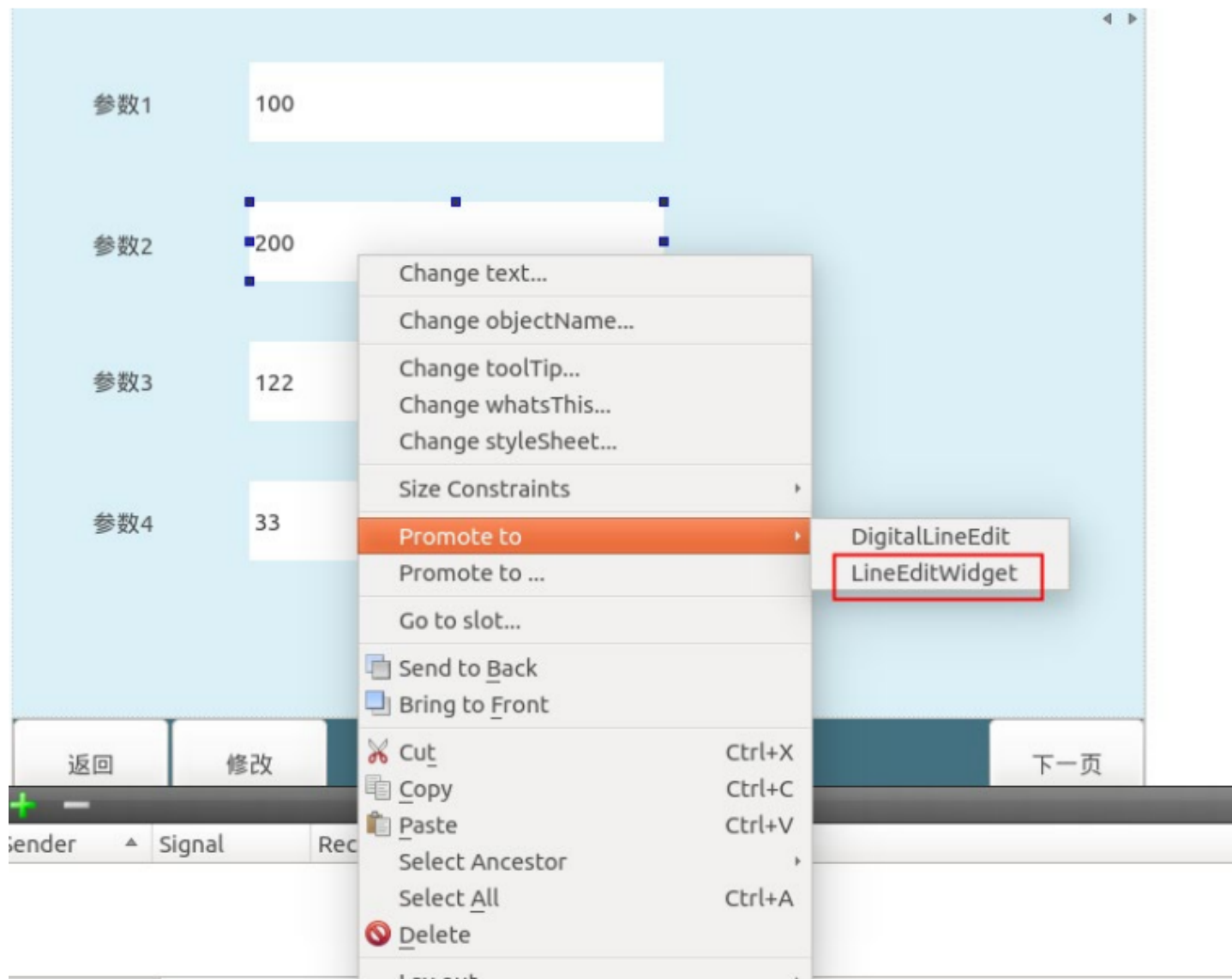
右侧树形结构中可以看到该控件 Classs 属性变为 DigitLineEdit



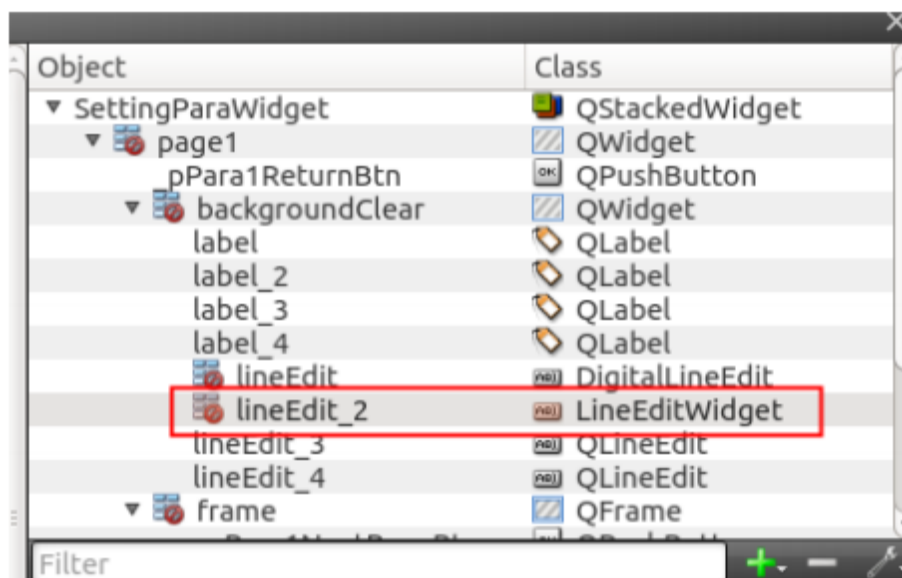
程序运行后控件效果，单击控件会弹出数字键盘：



4.lineeditwidget.h 提供数字和字符输入框 支持将 QLineEdit 控件提升为数字与字符输入框 提升方法：右键选择一个 QLineEdit 控 件 --->Promote to--->lineEditWidget



右侧树形结构中可以看到该控件 Classs 属性变为 QLineEditWidget

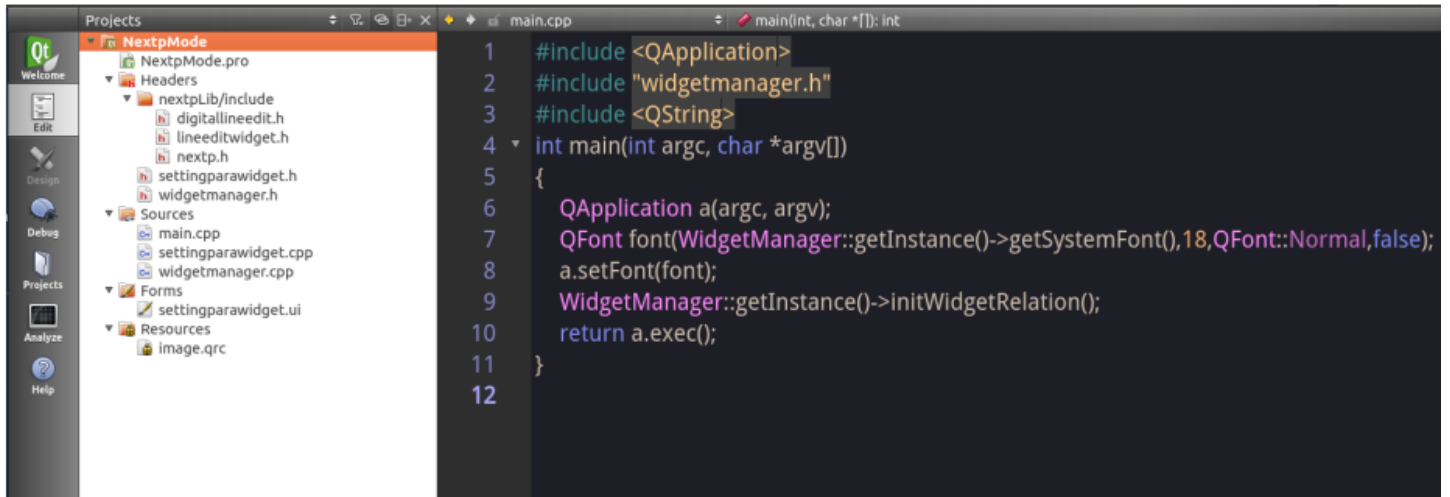


程序运行后控件效果，单击控件会弹出数字与字母键盘：



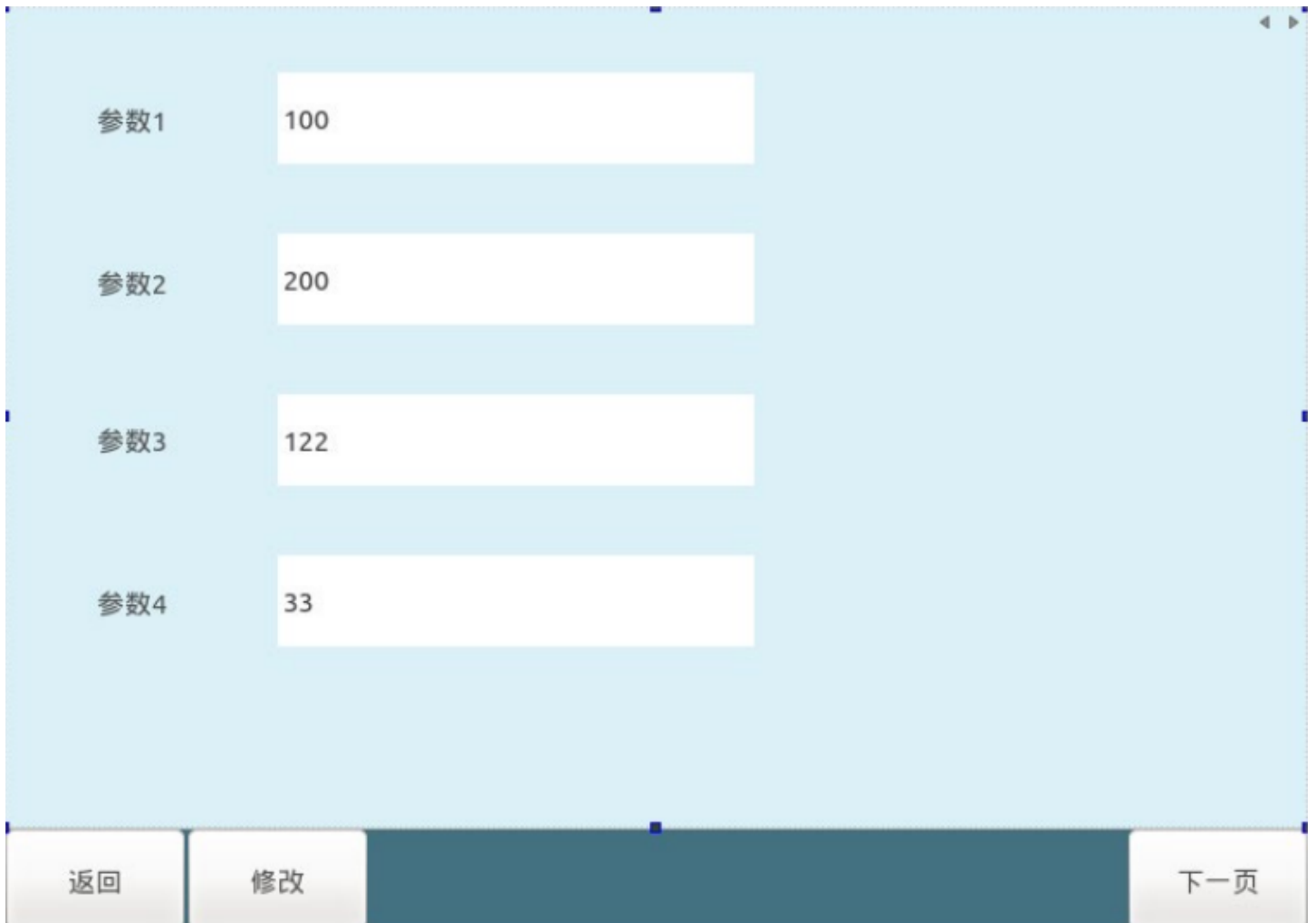
Demo 说明

1. Demo 结构图 demo 文件夹名称：NextpMode



2. Demo 文件类说明

2.1 settingparawidget.h settingparawidget.cpp settingparawidget.ui 三个文件为用户自定义窗体



2.2 widgetmanager.h widgetmanager.cpp 为管理类连接用户自定义窗体和静态库

2.2 静态库文件夹 nextplib 需要放置在 demod 的 NextpMode 文件夹下

2.3 运行 Demo （使用 QtCreator 直接打开 NextpMode 文件夹下的 NextpMode.pro 文件 ）

运行 Demo 程序点击【操作员】> 选择管理员>输入密码 123456 登录



点击左侧【工艺】按钮> 用户 进入自定义窗体





点击修改按钮 可以修改参数 点击保存将发送参数到控制器



QtCreator 控制台会打印发送到控制器的数据


```

70 else
71 {
72     Json::Value root;
73     Json::FastWriter wt;
74     root["para1"] = ui->lineEdit->text().toInt();
75     root["para2"] = ui->lineEdit_2->text().toString();
76     root["para3"] = ui->lineEdit_4->text().toInt();
77     root["para4"] = ui->lineEdit_3->text().toString();
78
79     root["robot"] = 1;
80     root["booldata"] = true;
81     root["data"] = 1.1;
82     root["name"] = "nihao";
83     Nextp::getInstance()->sendMessage(Nextp::GetFirstUserCommand,QByteArray().append(QString::fromStdString(wt.write(root))));
84
85
86     ui->_pParamodifyBtn->setText("修改");

```

Find: Replace with: Replace Replace & Find Replace All Advanced...

Application Output

NextpMode

删除用户窗体
 创建 用户自定义页面
 MaskWidget::show
 MaskWidget::hide
 "发送指令-->控制器 : {0x9201},{\"booldata\":true,\"data\":1.10,\"name\":\"nihao\",\"para1\":100,\"para2\":\"200\",\"para3\":122,\"para4\":\"33\",\"robot\":1}
 "

```

47
48 void WidgetManager::slot_receiveMessage(const quint16 &command,const QByteArray &data)
49 {
50     QString datastr = QString("接收控制器指令--> : [0x")+QString::number(command,16).toUpper()+QString(")");
51     qDebug()<<datastr;
52 }
53

```

Application Output

NextpMode

创建 用户自定义页面
 "发送指令-->控制器 : {0x9200},{\"booldata\":true,\"data\":1.10,\"name\":\"nihao\",\"para1\":100,\"para2\":\"200\",\"para3\":122,\"para4\":\"33\",\"robot\":1}
 "
 "接收控制器指令--> : [0x9201},{\"booldata\":true,\"data\":1.10,\"name\":\"nihao\",\"para1\":100,\"para2\":\"200\",\"para3\":122,\"para4\":\"33\",\"robot\":1}
 "

如果调用函数 void hideTechnologyToolbuttons();会隐藏除工艺在主界面上自定义按钮以外的其他工艺按钮



示例

包含示教器上硬件按钮以及滑轮的控制示例、自定义指令的示例

硬件接口

示教器上存在着一部分按钮和旋钮，下面示例演示如何获取到他们的状态：

建立一个Qt窗口应用

mainwindow.h

```
1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3  #include <QMainWindow>
4  #include <QSocketNotifier>
5  namespace Ui {
6  class MainWindow;
7  }
8  class MainWindow : public QMainWindow
9  {
10     Q_OBJECT
11 public:
```

```

12     explicit MainWindow(QWidget *parent = 0);
13     ~MainWindow();
14 private slots:
15     void onDataAvailable();
16     void getKeyValue();
17     void getEnableValue();
18 private:
19     Ui::MainWindow *ui;
20     int switchfd = -1;
21     QSocketNotifier *switch_notifier;
22     int buttonfd = -1;
23     QSocketNotifier *button_notifier;
24     int enablefd = -1; // 使能
25     QSocketNotifier *enable_notifier;
26 };
27 #endif // MAINWINDOW_H

```

mainwindow.cpp

```

1  #include "mainwindow.h"
2  #include "ui_mainwindow.h"
3  #include <stdio.h>
4  #include <fcntl.h>
5  #include <unistd.h>
6  #include <errno.h>
7  #include <unistd.h>
8  #include <linux/input.h>
9  #include <QFile>
10 #include <QDebug>
11 #include <QTime>
12 MainWindow::MainWindow(QWidget *parent) :
13     QMainWindow(parent),
14     ui(new Ui::MainWindow)
15 {
16     ui->setupUi(this);
17     // 打开旋钮设备文件
18     switchfd = ::open("/dev/buttons", O_RDONLY | O_NONBLOCK);

```

```
19     if (switchfd < 0) {
20         qCritical() << "Failed to open device:" << strerror(errno);
21         QApplication::exit(1);
22         return;
23     }
24     switch_notifier = new QSocketNotifier(switchfd, QSocketNotifier::Read, this);
25     connect(switch_notifier, SIGNAL(activated(int)), this, SLOT(onDataAvailable
26         ()));
27     //打开按键设备文件
28     buttonfd = ::open("/dev/input/event0", O_RDWR);
29     if (buttonfd < 0) {
30         qCritical() << "Failed to open device:" << strerror(errno);
31         QApplication::exit(1);
32         return;
33     }
34     button_notifier = new QSocketNotifier(buttonfd, QSocketNotifier::Read, this);
35     connect(button_notifier, SIGNAL(activated(int)), this, SLOT(getKeyValue()));
36     //打开使能设备文件
37     enablefd = ::open("/dev/buttonstop", O_RDWR);
38     if (enablefd < 0) {
39         qCritical() << "Failed to open device:" << strerror(errno);
40         QApplication::exit(1);
41         return;
42     }
43     enable_notifier = new QSocketNotifier(enablefd, QSocketNotifier::Read, this);
44     connect(enable_notifier, SIGNAL(activated(int)), this, SLOT(getEnableValue
45         ()));
46 }
47 MainWindow::~MainWindow()
48 {
49     delete ui;
50     if (switchfd >= 0) ::close(switchfd);
51     if (buttonfd >= 0) ::close(buttonfd);
52     if (enablefd >= 0) ::close(enablefd);
53 }
54 void MainWindow::onDataAvailable()
55 {
56     char switchbuttons[8]={'0','0','0','0','0','0','0','0'};
57     ssize_t bytesRead;
58     // 读取设备数据
```

```
57     while ((bytesRead = ::read(switchfd, switchbuttons, sizeof(switchbuttons))) >
0) {
58         qDebug() << "Data received:" << QByteArray(switchbuttons, bytesRead);
59         QString CurMode = "";
60         QString teachMode="00001000";//示教模式
61         QString runMode="00000100";//运行模式
62         QString remoteMode="00010000";//远程模式
63         for(unsigned int i=0;i<sizeof(switchbuttons);i++)
64         {
65             CurMode.append(QString(QChar(switchbuttons[i])));
66         }
67         qDebug() << CurMode;
68         if(CurMode == teachMode)
69         {
70             qDebug() << "示教模式";
71         }
72         else if(CurMode == runMode)
73         {
74             qDebug() << "运行模式";
75         }
76         else if(CurMode == remoteMode)
77         {
78             qDebug() << "远程模式";
79         }
80         else
81         {
82             //向上 00100000
83             //向下 01000000
84             if(CurMode == QString("00100000"))
85             {
86                 //上转
87                 qDebug() << "侧边滚轮向上";
88             }
89             else if(CurMode == QString("01000000"))
90             {
91                 qDebug() << "侧边滚轮向下";
92             }
93         }
94     }
```

```

95     if (bytesRead < 0 && errno != EAGAIN) {
96         qWarning() << "Read error:" << strerror(errno);
97     }
98 }
99 void MainWindow::getKeyValue()
100 {
101     struct input_event keyBuf;
102     unsigned int bytesRead = ::read(buttonfd,&keyBuf, sizeof(keyBuf));
103     if(bytesRead < sizeof(keyBuf))
104     {
105         return;
106     }
107     if(keyBuf.type==EV_KEY)
108     {
109         if(keyBuf.value==2)
110         {
111             return;
112         }
113         qDebug()<<"keyBuf.code = "<<keyBuf.code<<"keyBuf.value = "<<keyBuf.value<
<"time = "<<QTime::currentTime().toString("hh:mm:ss.zzz")<<endl;
114     }
115 }
116 void MainWindow::getEnableValue()
117 {
118     char enablebuttons[8]={'1','1','1','1','1','1','1','1'};
119     if(::read(enablefd,enablebuttons,sizeof(enablebutton
s))
120     {
121         return;
122     }
123     QString CurEnable = "";
124     for(unsigned int i=0;i<sizeof(enablebuttons);i++)
125     {
126         CurEnable.append(QString(QChar(enablebuttons[i])));
127     }
128     if(CurEnable == "11000000" ||CurEnable == "10000000" ||CurEnable == "0100000
0")
129     {
130         //上电
131         qDebug() << "上电";

```

```

132     }
133     else if(CurEnable == "00000000")
134     {
135         //下电
136         qDebug() << "下电";
137     }
138 }

```

自定义指令

在TeachPendantDemo中，建立三个文件

- my_custom_command.h
- my_custom_command.cpp
- my_custom_command.ui

在widget_mannager.cpp中将signal_userdefine_cmd_init与signal_userdefine_cmd_alter这两个信号和我们的界面绑定起来

```

1  WidgetManager::WidgetManager(QObject *parent) :
2      QObject(parent)
3  {
4      Nextp::getInstance();
5      // 注册my_custom_widget界面
6      connect(Nextp::getInstance(), SIGNAL(signal_openWidget()), this, SLOT(slot_openMyCustomWidget()));
7      // 注册my_custom_command界面
8      connect(Nextp::getInstance(), SIGNAL(signal_userdefine_cmd_init(int)), this, SLOT(slot_openMyCustomCommand(int)));
9      connect(Nextp::getInstance(), SIGNAL(signal_userdefine_cmd_alter(int, QString, QString)), this, SLOT(slot_modifyMyCustomCommand(int, QString, QString)));
10     // 注册关闭界面的信号,收到这个系统信号需要关闭所有custom界面
11     connect(Nextp::getInstance(), SIGNAL(signal_closeWidget()), this, SLOT(slot_closeAllCustomWidget()));
12     // 注册接收到控制器消息的回调
13     connect(Nextp::getInstance(), SIGNAL(signal_receiveMessage(const quint16 &, const QByteArray &)), this, SLOT(slot_receiveMessage(const quint16 &, const QByteArray &)));

```

```
14 }
```

```
1 void WidgetManager::slot_openMyCustomCommand(int)
2 {
3     Nextp::getInstance()->setWidgetParentLocation((QWidget*)MyCustomCommand::getI
    nstance(),86,96);
4     MyCustomCommand::getInstance()->show();
5     MyCustomCommand::getInstance()->raise();
6     Nextp::getInstance()->widgetShowFinish();
7 }
```

第一个信号是绑定打开事件，第二个信号是绑定修改事件。

在实现slot_openMyCustomCommand是打开一个全新的custom_commamd.ui

在实现slot_modifyMyCustomCommand是打开一个界面，并根据int,QString,QString这三个参数，初始化界面。